

Application 3 — Wallet-Rooted Device Certificate Issuance

PROVISIONAL PATENT APPLICATION

WALLET-ROOTED DEVICE CERTIFICATE ISSUANCE WITH OFFLINE CLAIM VERIFICATION AND THRESHOLD-SIGNATURE CUSTODY

Inventor(s)

Omar Ramadan

Assignee

Blockcast, Inc.

Filing Date

To be assigned upon USPTO submission (package finalized July 2026)

FIELD OF THE INVENTION

The invention relates to public key infrastructure (PKI) for distributed device networks, and more particularly to methods for authorizing X.509 device certificate issuance using blockchain wallet signatures verified offline, binding wallet-derived identifiers into certificates whose subject keys are distinct device keys, and to client-authenticated TLS from browser-sandboxed devices using non-extractable keys.

BACKGROUND

Decentralized physical infrastructure networks (DePIN) enroll gateway devices operated by third parties. Two identity roots must be reconciled: (a) the operator's blockchain wallet (e.g., a Solana Ed25519 account), which anchors economic identity, rewards, and on-chain name claims; and (b) the device's transport identity used for mutually authenticated TLS (mTLS) to control-plane services.

Prior decentralized-PKI approaches either place the wallet key directly in the certificate (making the high-value wallet key the routinely exercised TLS key — a custody downgrade), or require the certificate authority to query a blockchain at issuance time (adding availability and latency coupling to chain RPC). Separately, browser-sandboxed devices (e.g., Isolated Web Apps) cannot present TLS client certificates: platform fetch and WebTransport APIs provide no client-certificate attachment, and in-browser TLS stacks compiled to WebAssembly have provided server-side roles only.

SUMMARY OF THE INVENTION

A two-layer identity architecture in which a blockchain wallet AUTHORIZES certificate issuance but never becomes the certified key:

1. A registration authority (RA) verifies a wallet signature over a canonical claim message and mints a compact, expiring claim token (EdDSA-signed JWT) attesting the wallet's name claim.
2. The device places the claim token in a certificate signing request (CSR) attribute alongside a proof of possession of a distinct device key (Ed25519).
3. The certifier verifies the claim token OFFLINE against the RA's public key — no blockchain RPC at issuance — verifies the device attestation, and issues a short-lived certificate over the DEVICE public key, binding a wallet-derived identifier as an ADDITIVE subject alternative name of the form {logical_id}.{wallet_slug}.{suffix}, where `wallet_slug = base32(truncate(sha256(wallet_pubkey), 10))`.
4. Because every wallet-signature verification site in the protocol is a pure function of (public key, message, signature), a threshold / multi-party-computation (MPC) signer producing byte-identical Ed25519 signatures substitutes transparently: custody of the wallet key can be upgraded to threshold custody with zero protocol change. The MPC signer signs the 32-byte SHA-256 digest of the challenge (plain Ed25519, not the pre-hashed Ed25519ph variant).

5. Device-key loss is recoverable through a wallet-signature-gated re-binding path: a fail-closed guard rejects a changed device key for a bound installation identifier, and an upsert re-bind is authorized only by a fresh wallet signature, rooting key rotation in the wallet.
6. For browser-sandboxed devices, client-authenticated TLS is achieved by a WebAssembly TLS client whose CertificateVerify signature is produced through a signing callback into the platform cryptography API (`crypto.subtle.sign`) over a NON-EXTRACTABLE key, transported over a raw TCP socket (Direct Sockets API) from a privileged web application — client-certificate mTLS on a platform that natively cannot attach client certificates, with key custody equivalent to a hardware-backed store (the private key never exists in exportable form).

DETAILED DESCRIPTION

1. Two-layer identity and enrollment flow (implemented)

Layer 1 (wallet / authorization): the operator's wallet signs a canonical claim message `CanonicalClaim(wallet, network, expiry, nonce)`. The RA verifies via `ed25519.Verify(wallet_pubkey, message, signature)` — base58 public key, base64 detached signature, no key material beyond the public key — and mints a claim token: an EdDSA JWT signed by the RA's claim signing key, carrying the wallet public key, the derived slug ("`w`" + `base32(sha256(wallet_pubkey)[:10])`), and an expiry.

Layer 2 (device / transport): the device generates an Ed25519 key pair and a PKCS#10 CSR (RFC 8410 Ed25519 subject key). The CSR carries the claim token as an attribute (implemented as `CSR.wallet_claim_token` in the certifier protocol). Device attestation is challenge-response: the certifier issues a nonce; the device returns `Ed25519(SHA256(nonce))`; verification is again pure (pubkey, digest, signature) with key type `SOFTWARE_ED25519_SHA256`.

Issuance: the certifier verifies the claim token offline against the RA public key (no chain access), verifies the challenge signature, and signs a certificate over the DEVICE public key. The wallet contributes only an additive DNS-form SAN `{logical_id}.{wallet_slug}.{suffix}`. Certificates are short-lived (on the order of 97 hours); renewal is re-bootstrap with the same device key. The binding chain is: `wallet_pubkey` → (SIWS account auth) → `author_id` → `gateway_id` → `{install_id (hardware UUID), device challenge key, operator key}`, with wallet and device key meeting only at `author_id` — the wallet key is never on the TLS path and the TLS key never appears on chain.

2. Threshold-signature (MPC) transparency

All five wallet-signature verification sites in the deployed system (sign-in-with-Solana session auth; operator name-claim mint; user wallet verification; relay upgrade authorization; verified-supply attestation) verify a detached Ed25519 signature against a public key and message with no access to secret material. A t-of-n threshold Ed25519 signer producing standard signatures is therefore indistinguishable to the protocol. The only integration point that touches the wallet key is the RA name-claim mint. Dependent method: the enrollment challenge is signed as a 32-byte SHA-256 digest so that MPC nodes exchange only the digest, never the challenge plaintext.

3. Wallet-gated device-key rotation and recovery

A registration guard fails closed when a known installation identifier presents a different device key (“install_id already bound to different challengeKey”), preventing silent device-key substitution. The underlying binding store supports authorized re-binding (upsert on conflict). The rotation method: the operator’s wallet signs a rotation authorization naming the installation identifier and the new device public key; the RA verifies it exactly as a name claim; the guard admits the re-bind only with a valid wallet rotation authorization. Idempotent re-registration with the SAME device key requires no authorization (fresh token issuance for the same binding).

4. Browser-sandboxed mTLS client with non-extractable keys

The device may be an Isolated Web App. Platform networking cannot attach client certificates. The invention provides: - a WebAssembly TLS 1.3 client (rustls with a client-certificate resolver) configured with the issued device certificate chain; - a signing-callback certificate resolver: instead of holding the private key, the resolver invokes `crypto.subtle.sign` over a WebCrypto `CryptoKey` generated NON-EXTRACTABLE, so the `CertificateVerify` signature is computed inside the browser’s key store and the private key never exists in serialized form (a software analog of a TEE-held key); - transport over the Direct Sockets `TCPSocket` API from the privileged application context, with HTTP/1.1 or HTTP/2 framed above the WASM TLS session; - an enrollment-transport seam such that the same client performs both RFC 7030 EST (`/cacerts`, `/simpleenroll`) and proprietary bootstrap protocols. An alternative embodiment holds the device key extractable (PKCS#8 in origin-private storage) during a migration period, with paragraph-4 custody as the target state.

5. Embodiments and variations

- The blockchain may be any Ed25519-account chain (Solana described); RSA/ECDSA wallets with corresponding verifiers are equivalent.
- The claim token may be any offline-verifiable signed assertion (JWT, CWT, SD-JWT).
- The wallet-derived SAN may be a URI SAN (e.g., spiffe://{domain}/wallet/{pubkey}) in place of or in addition to the DNS-form slug; a raw-public-key embodiment includes the wallet public key itself in a subject attribute.
- The certifier and RA may be co-located or separate; the RA may be a network operator portal; the certifier may be an EST server.
- Devices include set-top firmware, mobile SDKs, and browser IWAs.

APPENDIX — CLAIM SKETCH (for the non-provisional)

Independent (method): issuing a device certificate in a network where operators hold blockchain wallet keys, comprising: verifying, at an RA, a wallet signature over a canonical claim; minting an expiring signed claim token; receiving a CSR comprising (i) a device public key distinct from the wallet public key, (ii) the claim token, and (iii) a device attestation; verifying the claim token offline against the RA public key without blockchain access; and issuing a certificate over the device public key including a wallet-derived identifier as an additional subject alternative name.

Dependents: slug derivation by truncated hash + base32; threshold-Ed25519 wallet custody with byte-identical signatures (no protocol change); digest-only MPC signing; short-lived certificate + re-bootstrap renewal; fail-closed installation binding with wallet-signature-gated device-key re-bind; SIWS-authenticated account chain wallet→author→gateway→device.

Independent (system/CRM): the browser-sandboxed mTLS client of §4 (WASM TLS client + non-extractable-key signing callback + raw-socket transport), independently and in combination with the issuance method.

CROSS-REFERENCE

Related applications by the same inventor filed 2026-07-10: 64/109,453 (FEC methods for publish/subscribe media transport); 64/109,460 (broadcast-to-unicast media bridge). The present disclosure is an independent invention family (device identity/PKI); the certified device identity of this disclosure authenticates the transport sessions over which the media of the related applications is delivered.